

week11 연습과제

☼ Status	Not started
≡ Text	295~372

Word2Vec

- 단어 간의 유사성을 측정하기 위해 분포 가설을 기반으로 개발

분포 가설

- 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정
- 단어 간의 동시 발생 확률 분포를 이용해 단어 간의 유사성 측정
- 이런 가정으로 단어의 분산 표현 학습 가능
 - 분산 표현: 단어를 고차원 벡터 공간에 매핑 → 단어의 의미 담는 것
- 단어의 의미는 문맥상 분포적 특성을 통해 나타남
 - 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 된
- 이 방법으로 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계 극복
- 장점) 대량의 텍스트 데이터에서 단어 간의 관계를 파악하고 벡터 공간상에서 유사한 단어를 군집화 → 단어의 의미 정보 효과적 표현

단어 벡터화

- 희소 표현(sparse representation) → 원-핫 인코딩, TF-IDF
 - 대부분의 벡터 요소가 0으로 표현
 - 단어 사전의 크기가 커짐 → 벡터의 크기도 커짐 → 공간적 낭비 발생
 - 단어간 유사성 반영X, 벡터 간 유사성 계산 많은 비용 발생
- 밀집 표현(dense representation) → Word2Vec

- 단어를 고정된 크기의 실수 벡터로 표현 → 단어 사전의 크기 커지더라도 벡터의 크기 커지지 X
- 벡터 공간상에서 단어 간의 거리를 효과적으로 계산
- 효율적으로 공간 활용
- 밀집 표현된 벡터: 단어 임베딩 벡터(Word Embedding Vector)

CBow (Continuous Bag of Words)

- 주변에 있는 단어로 중간에 있는 단어 예측
- **중심 단어**: 예측해야 할 단어
- **주변 단어**: 예측에 사용되는 단어
- 몇개의 주변 단어 고려할 지 범위 → **윈도(Window)**
- 윈도를 이동해 가며 학습 → **슬라이딩 윈도(Sliding Window)**
- CBow의 학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

- CBow 모델 구조

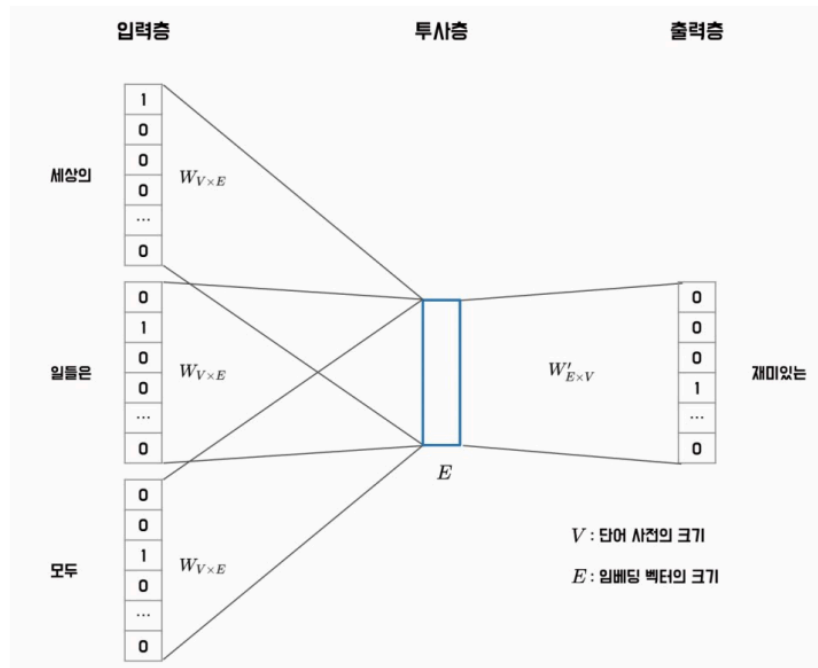


그림 6.5 CBOW 모델 구조

- 입력값 : 원-핫 벡터
- 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터 예측
- 입력 단어: 원-핫 벡터로 표현 - 투사층(Projection Layer)에 입력
 - 투사층: 원-핫 벡터의 인덱스(임베딩 벡터)를 반환하는 순람표 구조
- 투사층 통과 → 각 단어는 E 크기의 임베딩 벡터로 변환 → 평균 벡터 계산 → 가중치 행렬 곱함 → 소프트맥스 함수로 중심 단어 예측

Skip-gram

- 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델
- 중심 단어를 기준으로 양쪽으로 원도 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋 만들
- 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델 학습
- 학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

- 하나의 중심 단어를 통해 여러 개의 주변 단어 예측 → 더 많은 학습 데이터셋 추출 가능
- CBow보다 뛰어난 성능
- 모델 구조

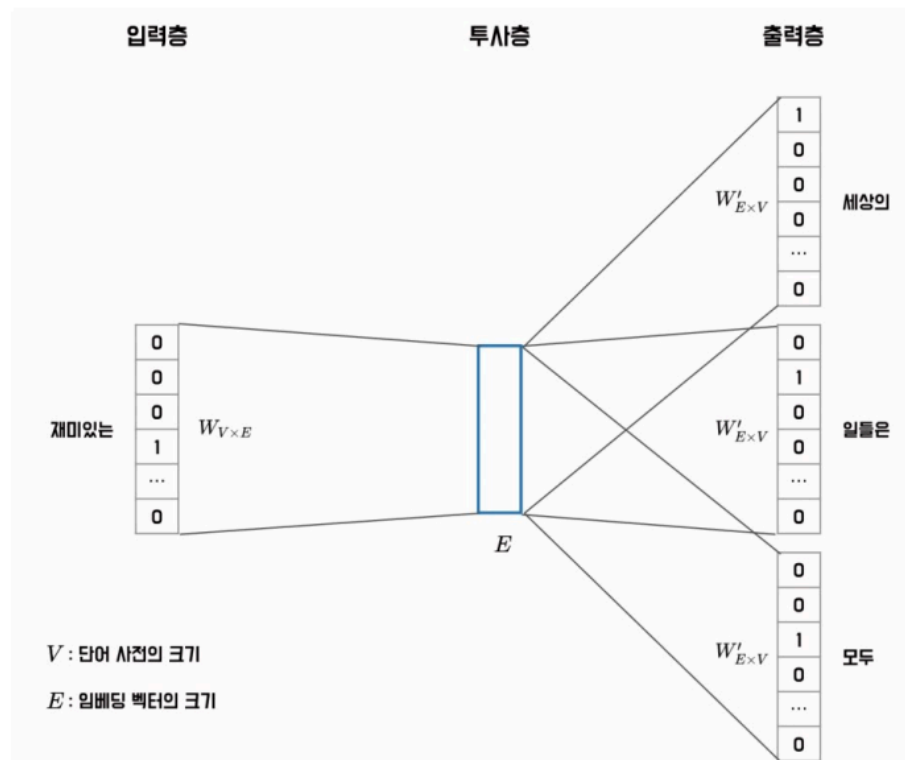


그림 6.7 Skip-gram 모델 구조

- 입력 단어의 원-핫 벡터를 투사층에 입력 → 해당 단어의 임베딩 벡터 가져옴
- → CBow와 마찬가지로...

- 소프트 맥스 연산: 모든 단어를 대상으로 내적 연산 수행
 - 말뭉치 크기 커짐 → 단어 사전의 크기도 커짐 → 학습 속도 느려지는 단점
 - ⇒ 계층적 소프트 맥스 / 네거티브 샘플링 기법

계층적 소프트맥스

- 출력층을 이진 트리 구조로 만들어 단어 예측
- 자주 등장하는 단어 - 상위노드, 드물게 등장 - 하위 노드
- 각 노드는 학습 가능한 벡터 가짐, 입력/내적 후 시그모이드 함수로 확률 계산
- 하나의 단어 확률: 루트→ 해당 단어까지의 경로 노드 확률 모두 곱함
- 특정 단어 학습할 때 해당 경로의 노드들만 업데이트 하면 됨
- 시간 복잡도
 - 일반 소프트맥스: $O(V)$
 - 계층적: $O(\log V)$

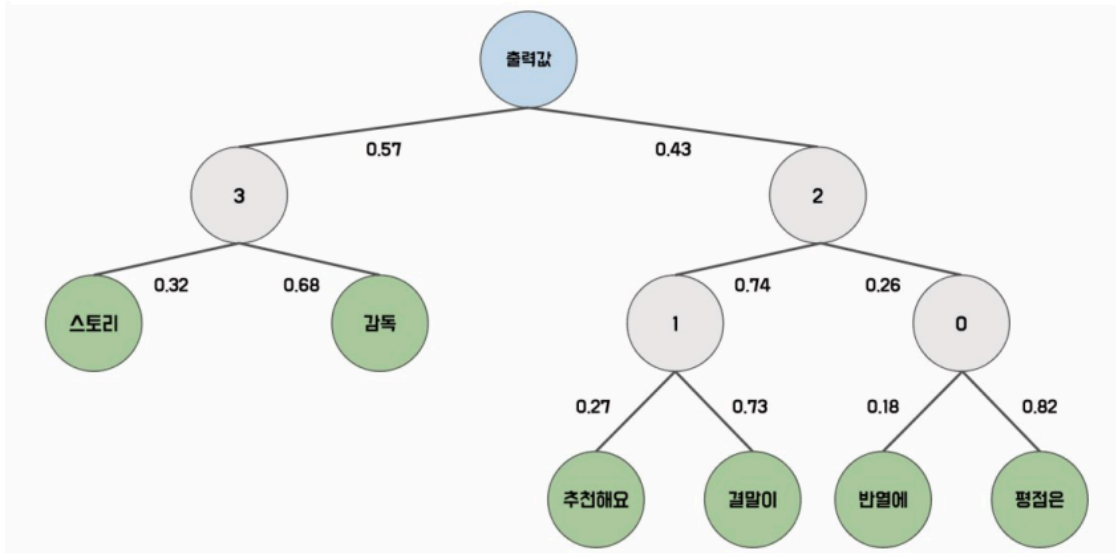


그림 6.8 계층적 소프트맥스 구조

네거티브 샘플링

- Word2Vec 모델에서 전체 단어에 대해 Softmax 계단하는 부담 줄이기 위한 확률적 샘플링 기법

- 학습 윈도 안에 등장하지 않은 단어들 뽑아 → Negative 단어로 설정
- 실제로 등장한 단어 쌍 - 1, 네거티브 샘플로 만든 단어 쌍 - 0 ⇒ 이진 분류
- 단어가 네거티브 샘플로 선택될 확률: 출현빈도 $f(w)$ 의 0.75제곱 정규화
- 각 단어 쌍은 입력 임베딩과 가중치의 내적→시그모이드
 - 1→ 확률 높아지도록
 - 0→ 확률 낮아지도록 가중치 업데이트

fastText

- Word2Vec과 유사한 방식으로 텍스트를 벡터로 변환
- 차이점: 단어를 그대로 벡터화X, 단어를 여러 하위 단위 (Subword)로 분해해 학습
 - 하위단어: N-gram 방식으로 생성 <>
- 분해된 하위 단어 - 각각 고유한 벡터 가짐
- 최종 단어 벡터 - 하위 단어 벡터들의 합 or 평균
- 장점: 희귀 단어, 신조어에 강함
 - 형태가 비슷한 단어끼리 유사도 잘 반영
 - Word2Vec보다 정확도, 일반화 능력 높

입력 단어	< > 더하기	N-gram 분해	전체 단어 추가
서울특별시	<서울특별시>	<서울 서울특 울특별 특별시 별시>	<서울 서울특 울특별 특별시 별시> <서울특별시>

그림 6.11 fastText의 하위 단어 집합

하위 단어 생성 과정

1. 입력 토큰 양 끝에 <> 기호 붙여 시작/끝 위치 표시

2. N-gram으로 토큰을 하위 단어 집합으로 분해
3. 분해된 집합엔 토큰 자체도 포함됨
4. 각 하위단어는 고유 벡터 가짐

단어 임베딩 계산 방식

- 생성된 하위 단어 벡터들을 모두 합산 → 입력 단어의 최종 임베딩 만듦

fastText 특징

- 다양한 N-gram 적용으로 유연하고 정확한 하위 단어 집합 구성
- 같은 하위 단어를 공유하는 단어들끼리 의미 정보를 공유해 학습
 - 비슷한 단어는 비슷하 임베딩 벡터
- 단어가 말뭉치에 없어도 하위 단어 조합으로 OOV 단어 임베딩 계산 가능

순환 신경망

- 순서가 있는 연속형 데이터를 처리하는데 적합
- 각 시점의 데이터가 이전 시점의 영향을 받는다는 특성 활용
- Sequence data
 - 주가 데이터등
 - 과거 시점의 값에 의존
 - 자연어 문장 - 단어들이 순서대로 등장, 이전 단어들의 문맥에 영향 받음
 - 긴 문장 - 단어간 강한 상관관

순환 신경망

목적: 연속적/순서가 있는 데이터를 처리하기 위해 만들어진 인공 신경망

- 과거에 처리한 데이터를 내부에 저장 (은닉 상태)
- 현재 입력과 함께 활용 → 현재 상태 예측

활용 분야

- 시계열 데이터
- 자연어 처리
- 음성 인식
- 길이가 가변적, 순서에 의미 0

동작 방식

- 데이터를 시점 순서대로 입력받아 처리
- 각 시점마다 은닉 상태 유지, 이전 상태 기반으로 다음 상태 계산
 - RNN Cell: h_{t-1} , x_t 받아서 h_t 생성
 - 재귀적 구조

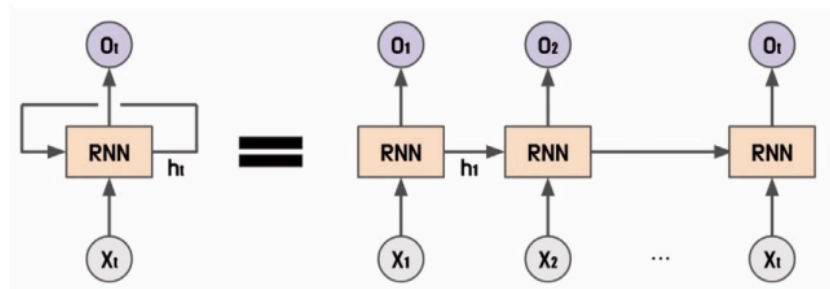


그림 6.12 순환 신경망의 셀

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$
$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$
$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- 구조
 - 단순 순환 구조, 일대다 구조, 다대일 구조, 다대다 구조

일대다 구조

- 하나의 입력 시퀀스 → 여러 개의 출력값
- 품사 태깅, 이미지 캡셔닝
- 출력 시퀀스의 길이를 미리 알고 있어야함
 - 출력 시퀀스 길이를 예측하는 모델 사용

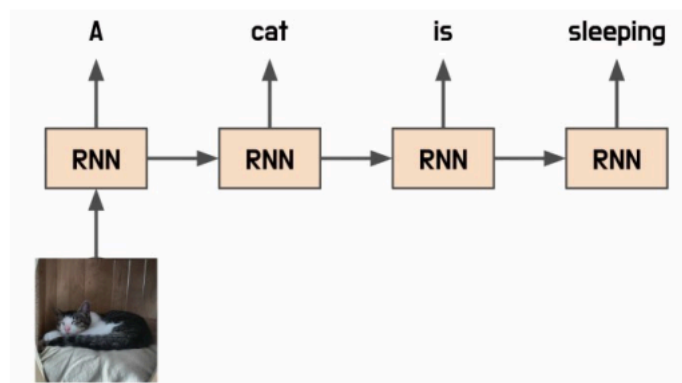


그림 6.13 순환 신경망의 일대다 구조

다대일 구조

- 여러 개의 입력 시퀀스 → 하나의 출력값
- 감성 분류, 문장 분류, 자연어 추론

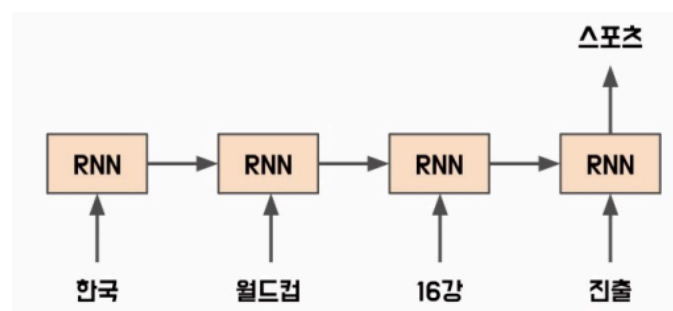


그림 6.14 순환 신경망의 다대일 구조

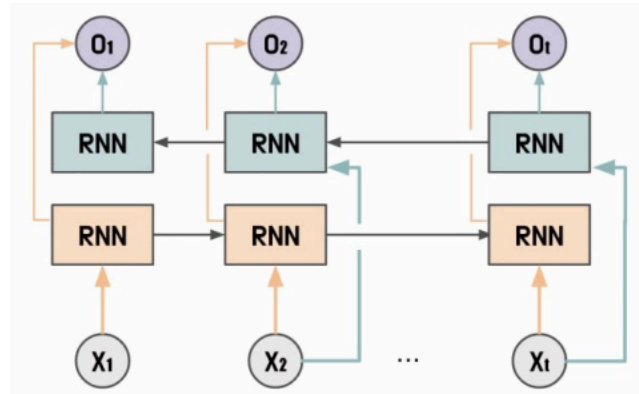


그림 6.16 양방향 순환 신경망 구조

다중 순환 신경망

- 여러 개의 RNN 층을 위로 쌓아 구성한 모델
- 각 층: 서로 다른 수준의 정보 처리하도록 설계
- 첫번째 RNN층의 출력 → 다음 RNN 층의 입력
- 층을 여러개 쌓으면 → 더 다양한 특징/복잡한 패턴
- 장점: 더 깊은 구조 → 복잡한 패턴 학습, 다양한 특징 추출
- 단점: 층이 많아질 수록 → 학습 시간 증가, 기울기 소실 문제 발생 가능성

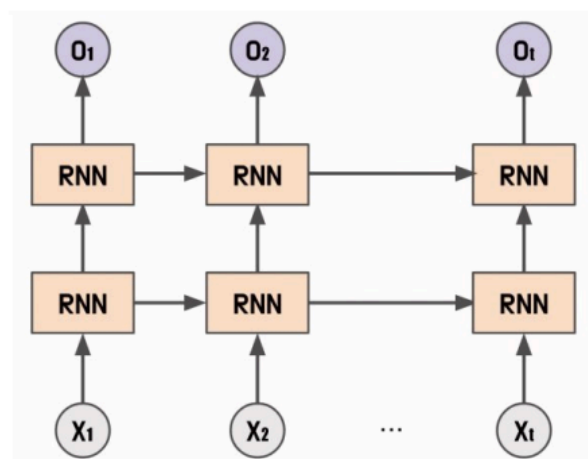


그림 6.17 다중 순환 신경망 구조

장단기 메모리 (Long Short-Term Memory)

- RNN의 기억력 부족/기울기 소실 문제 해결

RNN의 한계

- 먼 과거 정보는 잘 기억하지 못함
- 기울기 소실 / 기울기 폭주 문제
- ⇒ 장기 의존성 처리하기 어려움

해결 방식

- memory cell, gate 구조 도입
 - 먼 과거의 중요한 정보를 장기간 저장
 - 필요없는 정보 제거
 - 기울기 소실 문제 크게 완화

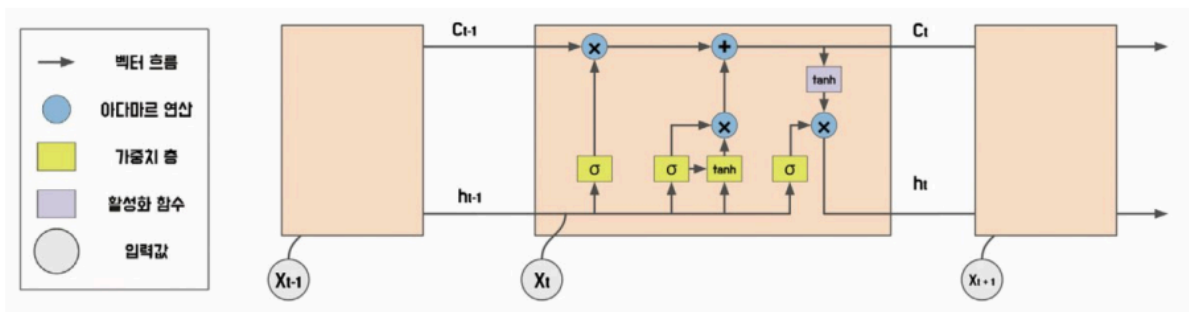


그림 6.18 장단기 메모리 구조

- 셀 상태: 정보를 저장하고 유지하는 메모리 역할
- 망각 게이트: 이전 셀 상태에서 어떤 정보 삭제할 지 결정
- 입력 게이트: 새로운 정보를 어디에 추가할지 결정
- 출력 게이트: 셀 상태 중 어떤 정보를 출력할지 결정

장단기 메모리 구조

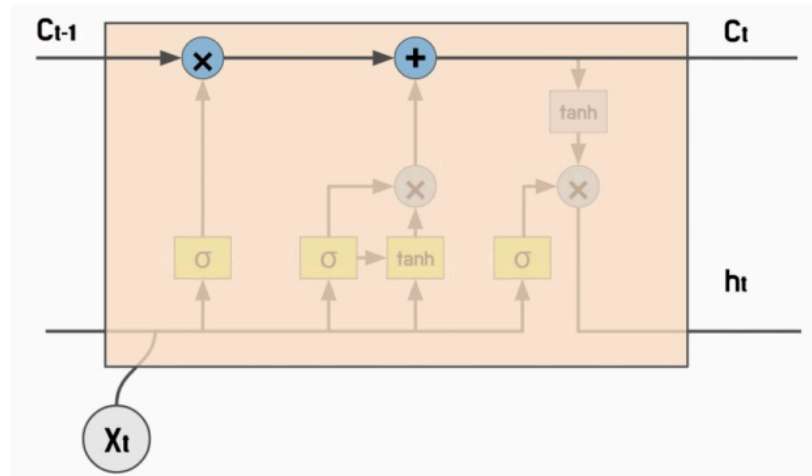


그림 6.19 장단기 메모리 셀 상태

- 셀 상태와 망각/입력/출력 게이트로 → 어떤 정보를 유지/삭제/추가/출력할지 결정
 - 입력: x_t , h_{t-1} 사용
 - Cell state: 메모리를 장기적으로 저장하는 경로
 - 출력값을 직접 만드는 용도 X
 - 망각 게이트: 무엇을 버릴 지 결정
 - 입력 게이트: 어떤 새로운 정보를 추가할지 결정
 - 출력 게이트: 셀 상태 중 어떤 일부를 출력으로 사용할지 결정
 - 게이트 조절 →
 - 필요한 정보는 오래 기억
 - 불필요한 정보는 제거
 - 기울기 소실 문제 완화
- ⇒ 장기 의존성 해결

망각 게이트

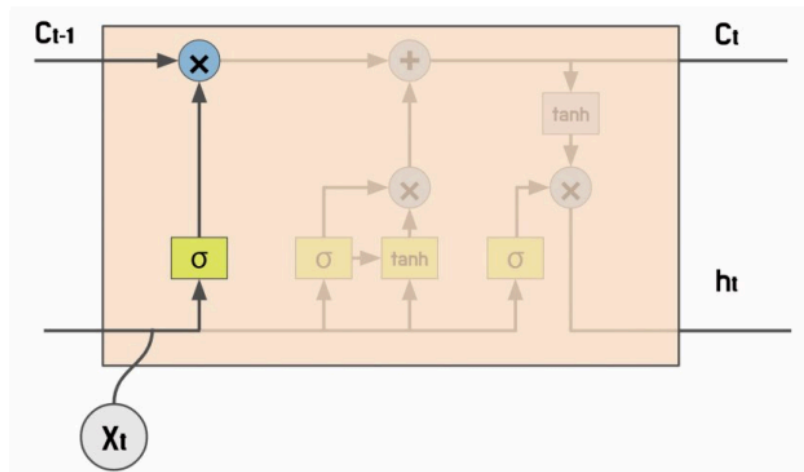


그림 6.20 장단기 메모리의 망각 게이트

입력: h_{t-1} , x_t

- 이전 시점의 메모리 셀에서 어떤 정보를 삭제할지 결정
- 출력값이
 - 1에 가까울 수록 → 정보를 많이 유지
 - 0에 가까울 수록 → 정보를 많이 삭제
 - 1 = 정보 삭제 X
 - 0 = 모든 정보 삭제
- 출력값: 메모리 셀 계산에 사용되는 가중치 역할

기억 게이트

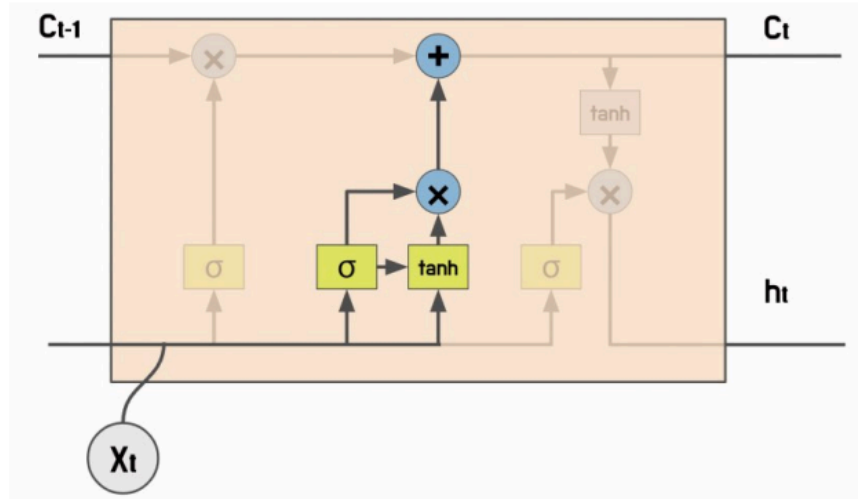


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_x^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_x^{(i)}h_{t-1} + b^{(i)})$$

- g
 - 활성화 함수: tanh
 - 출력 범위 [-1, 1]
 - 새로운 후보 생성
- i
 - 활성화 함수: 시그모이드
 - 출력 범위 [0, 1]
 - g에서 생성된 새로운 정보를 얼마나 기억할지 결정 (가중치 역할)
- g는 정보의 범위를 [-1, 1]로 제한, but 얼마나 반영할지는 조절 x
- i가 g를 얼마나 기억할 건지 결정

출력 게이트

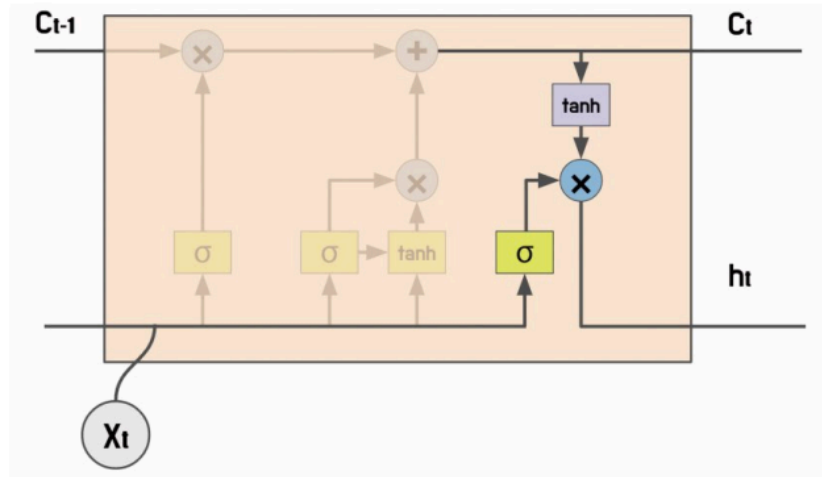


그림 6.22 장단기 메모리의 출력 게이트

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

- 역할: 현재 시점의 은닉 상태를 얼마나 출력할지 결정
- 메모리 셀에서 어떤 정보를 꺼낼지 제어하는 단계

합성곱 신경망(CNN)

- 컴퓨터 비전에서 이미지 인식을 위해 사용
- 입력 데이터에서 Local Features 추출하는 데 특화
- 특징
 - 합성곱 연산 → 입력 데이터의 특정 작은 영역에서 특징 추출
 - 여러 지역 특징을 조합해서 전역 특징 파악 가능
- 장점
 - 입력 길이와 관계없이 병렬 처리 가능
 - 가중치 수를 줄여 깊은 신경망 구성

합성곱 계층

- 입력 데이터와 필터를 합성곱 → 특징 추출
- 고차원 데이터의 지역 패턴 효과적으로 인식
- 필터를 모든 위치에 공유 → 매개변수 수 줄임 / 과대 적합 방지
- 위치와 상관없이 동일한 특징 추출 가능
- 여러 합성곱 계층 → 더 복잡하고 다양한 특징 학습

필터

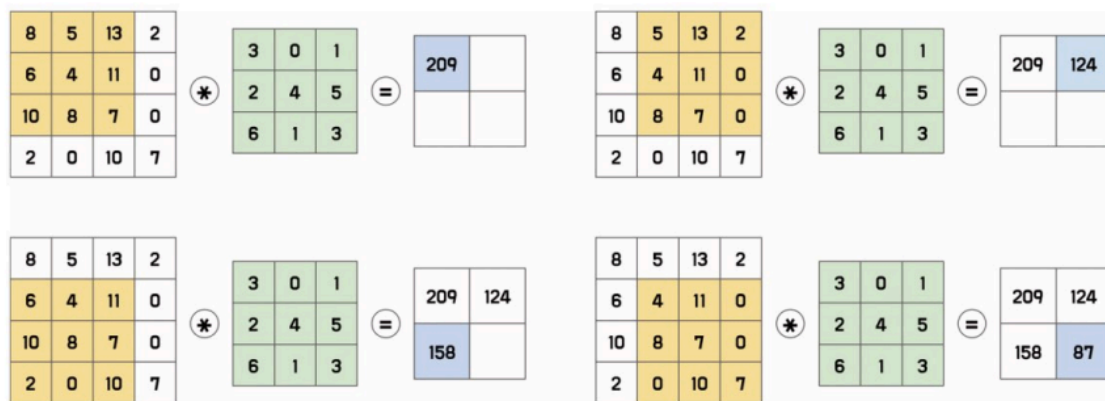


그림 6.24 특징 맵 추출 과정

- 정방향 행렬
- 입력 데이터 위를 움직이며 합성곱 연산 수행
- 필터의 가중치: 학습 과정에서 갱신
여러 필터로 다양한 특징 추출
- 입력 전체를 동일한 가중치로 스캔 → 위치와 무관하게 같은 형태의 패턴 인식 가능
- 생성된 특징 맵 → 다음 합성곱 계층의 입력으로 사용

패딩

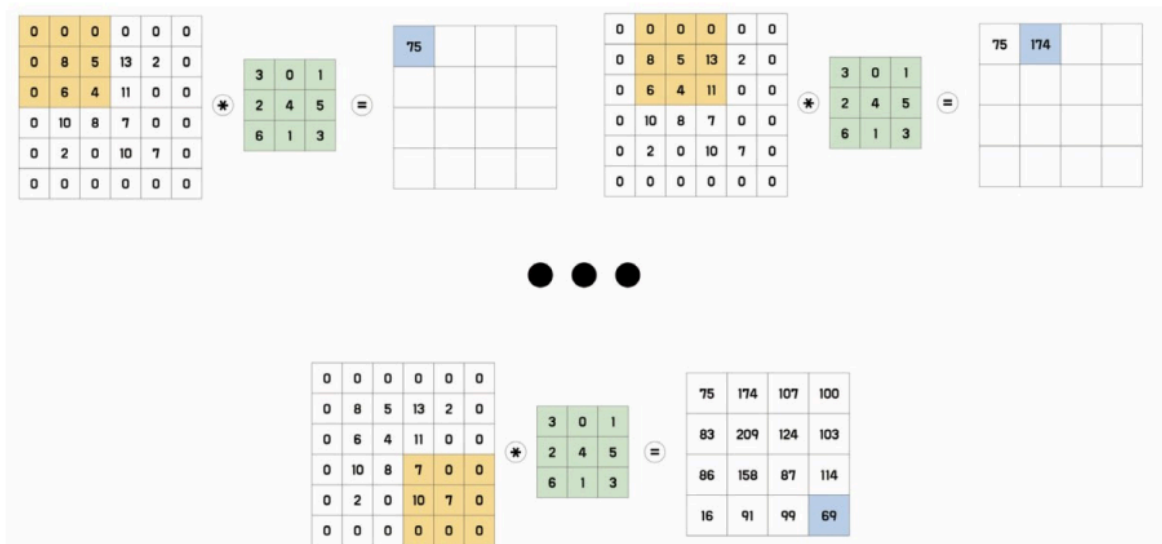


그림 6.25 제로 패딩

문제) 합성곱 수행시 출력 크기가 줄어드는 문제

- 가장자리 픽셀은 연산에 적게 참여 → 정보 학습이 불리

해결) 입력 가장자리에 0을 채워 넣음 ⇒ 제로 패딩

패딩 적용 → 출력 크기를 입력과 동일하게 유지 가능

- 작은 필터 사용해도 전체 이미지 정보 더 잘 반

간격

- 필터가 한 번에 이↖↗하는 크기
- Stride = 1 : 한 픽셀씩 이동 → 출력 크기가 크게 유지
- Stride ≥ 2 : 더 크게 이동 → 출력 크기가 작아짐
- Stride가 작으면 공간 정보를 잘 보존할 수 있음
- Stride가 크면 공간 정보 줄어듦, 모델 복잡도/매개변수 감소, 과대 적합 방지

채널

- 채널 수가 많을 수록
 - 더 다양한 특징 학습

- 모델의 표현력 증가
- 단점) 채널 수 증가
 - 매개변수/연산량 증가, 메모리 사용량 증가
 - 과대적합 위험 증가

팽창

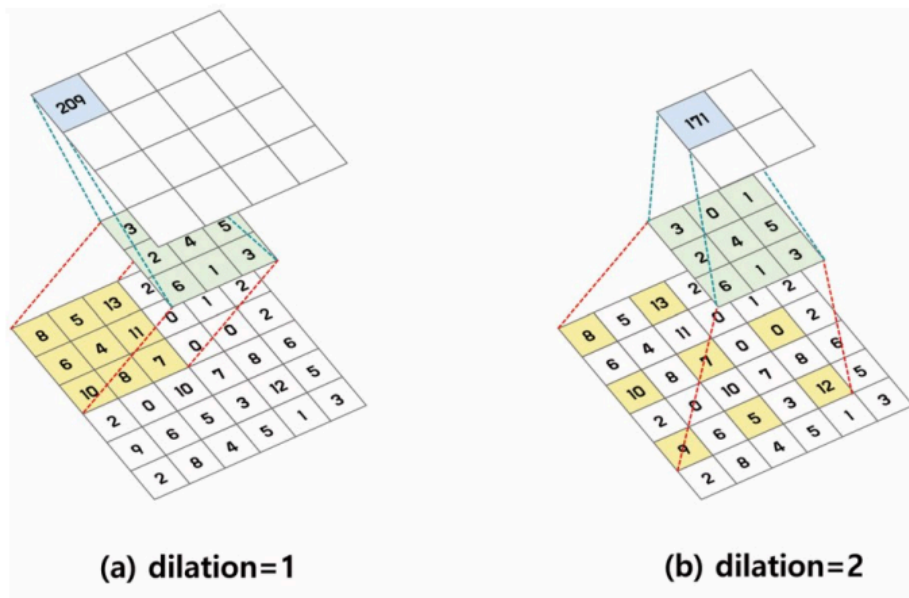


그림 6.26 팽창

- 필터와 입력 사이에 간격 - 더 넓은 영역을 보게 만드는 기법
- Dilation = 1 → 일반 합성곱
- Dilation > 1 → 입력 건너뛰며 연산 → 더 넓은 범위의 정보 반영
- 필터 크기 키우지 않고도 수용 영역 확대 가능 → 깊고 복잡한 모델 구성 가능
- 매개변수는 늘지 않지만 연산량 증가 가능
- Dilation 너무 크면 인접 픽셀 정보 잃어 공간 정보가 흐트러질 수 있음

합성곱 계층 클래스

- Conv2d
 - 입력 채널 → 출력 채널로 합성곱 연산 수행하는 2D 합성곱 계층

- 내부적으로 필터 이용해 특징맵 생성
- 입력의 공간 정보를 유지/축소하며 특징 추출
- stride, padding, dilation 조절로 출력 크기 유연하게 제어 가능
- groups로 연산량/파라미터 감소 가능

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$

$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

활성화 맵

- 합성곱 계층이 만든 특징 맵에 활성화 함수를 적용한 결과
 - 비선형성 부여, 특징 강조
1. 입력 이미지 + 필터 → 특징 맵 생성
 2. 특징 맵에 활성화 함수 적용 → 0 이하 값은 0, 양수는 그대로
 3. 활성화 맵이 다음 계층 입력이 됨
 - 모델이 어떤 부분을 주목하는지 확인

풀링

- 특징 맵의 크기를 줄이는 연산
- 합성곱 계층 뒤에 적용
- 연산량 감소 + 특징 압축 + 공간적 정보 요약 효과

- Max Pooling
 - 필터 영역에서 가장 큰 값만 선택
 - 중요한 특징 잘 보존

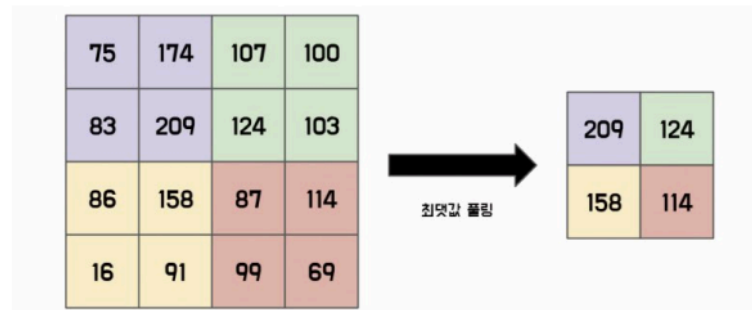


그림 6.27 최대값 풀링

- Average Pooling
 - 필터 영역 값들의 평균 사용
 - 부드러운 특징 요약

완전 연결 계층

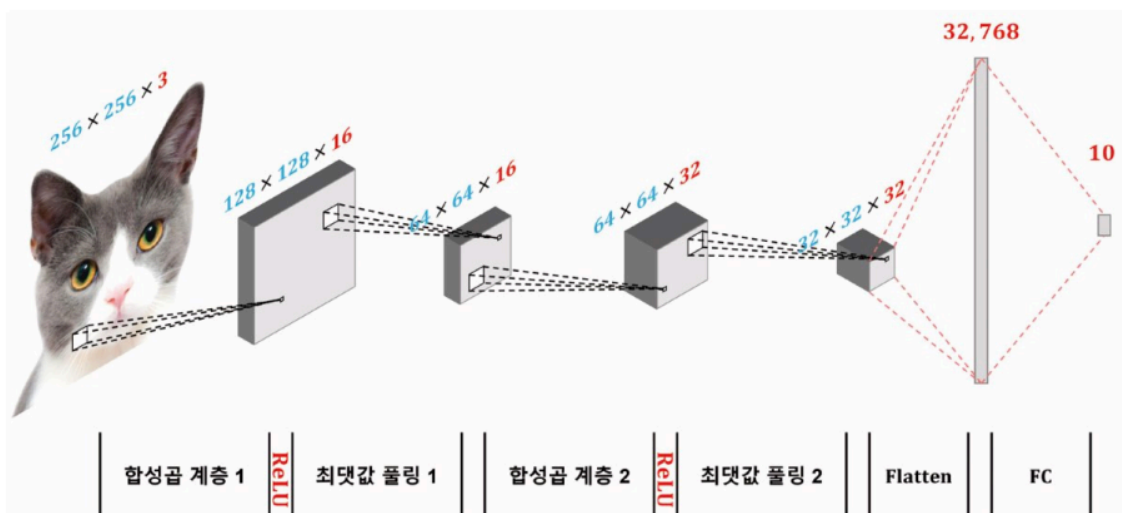


그림 6.28 합성곱 모델 구조 시각화

- 모든 입력 노드가 모든 출력 노드와 연결된 계층

- 입력-출력 간 모든 관계 학습 가능, 출력 노드 수로 모델 복잡도 조절 가능
- 합성곱/풀링 후의 3D 특징 맵을 1D로 평탄화 → 완전 연결층에 입력
- 공간 정보 무시하고 전체 특징 종합해 처리
- 최종 분류 담당
- 마지막 완전 연결 계층 출력에 소프트맥스/시그모이드 적용해 분류 모델 완